2.4.3 Executable specifications: automating the acceptance criteria

Expressing your requirements in the form of structured examples provides many benefits. Examples tend to make great starting points for discussions of business needs and expectations, and they make it much easier to flush out misunderstandings and invalid assumptions than do more abstract specifications. Another major advantage of this approach is that it makes it easier to automate the requirements in the form of acceptance tests.

Now that you've got your development environment set up, it's time to automate the example we discussed in the previous sections. JBehave, like many BDD tools, uses a special language to represent executable specifications in a structured but still very readable form. In JBehave, you could express the scenario as follows.

Listing 2.1 Acceptance criteria expressed in JBehave

```
Find out what time the next trains for my destination station leave

Narrative:
In order to plan my trips more effectively
As a commuter
I want to know the next trains going to my destination

Scenario: Find the optimal itinerary between stations on the same line
Given Western line trains from Emu Plains leave Parramatta
for Town Hall at 7:58, 8:00, 8:02, 8:11, 8:14, 8:21
When I want to travel from Parramatta to Town Hall at 8:00
Then I should be told about the trains at: 8:02, 8:11, 8:14
```

1 Story title
2 Freeform story description
3 Scenario
4 Preconditions
5 Action under test
6 Expected outcome

This is little more than a structured version of the example we discussed earlier. You start off with the story description 1 and 2. The `Scenario` keyword 3 marks the start of each new scenario. The keywords `Given` 4, `When` 5, and `Then` 6 introduce the various parts of each scenario.

In JBehave, scenarios are grouped by story and are stored in files with a .story suffix.[10] As you can see in figure 2.3, the file containing this story definition is called find_next_train_departures.story.

You could place all your .story files directly in the stories directory, but this becomes unwieldy when you start to have a large number of story files. Instead, it's best to group the stories into high-level functional groups. For example, as this project progresses, you might end up with directories such as

- itineraries (itinerary calculations and timetable information)
- commuters (personalized trip data for commuters)
- notifications (delay notifications for commuters)

[10] This is slightly different from Cucumber and Cucumber-based tools, which use .feature files. We'll come back to this difference and its implications in chapter 5.